Tipos, tiempo y pulsadores

Tres temas: el **tamaño de los tipos de datos**, medir el **tiempo sin bloquear** con `millis()` y leer un **pulsador** sin errores.

Hardware · Software · código · Regla · decisión · Error · ojo · ◇ decisión

1 Tiempo sin bloquear: millis()

`millis()` = milisegundos desde el arranque

1 — Leer el tiempo
En cada vuelta del `loop()`.
`ahora = millis()`

2 — ahora − tAnterior ≥ PERIODO

Sí → **Actuar y anotar**
Hacer la tarea y seguir.
`tAnterior += PERIODO`

No → **3 — Seguir con lo demás**
Leer botones, etc.

↻ vuelve al paso 1: el `loop()` nunca se detiene

delay()
HACE Detiene el programa
PROBLEMA No lee nada mientras espera

vs

millis()
HACE Solo mira el tiempo
VENTAJA El `loop()` sigue corriendo

Regla: `ahora - tAnterior >= PERIODO`, con variables `uint32_t`.

CÓDIGO: EL PATRÓN MILLIS()

```
uint32_t tAnterior = 0;
const uint32_t PERIODO = 500;      // ms

void loop() {
  uint32_t ahora = millis();
  if (ahora - tAnterior >= PERIODO) {
    tAnterior += PERIODO;
    // hacer la tarea
  }
}
```

OJO
La resta funciona aunque el contador dé la vuelta. ✘ `millis() > tAnterior + PERIODO` falla cuando la suma da la vuelta.

2 Tipos de datos: tamaño fijo

int no mide lo mismo en todas las placas

Arduino Uno (AVR)
`INT` 2 bytes
`PUNTERO` 2 bytes

VS

ESP32
`INT` 4 bytes
`PUNTERO` 4 bytes

Para que el tamaño no cambie: usar `uint8_t`, `uint16_t`, `uint32_t`.

`uint8_t`
8 bits, sin signo
`0 a 255`

`uint16_t`
16 bits, sin signo
`0 a 65 535`

`uint32_t`
32 bits, sin signo
`0 a 4 294 967 295`

3 Desbordamiento

pasar del máximo del tipo

1. `uint8_t c = 253;` — Máximo del tipo: 255. → `c = 253`
2. `c++` — Sube uno. → `c = 254, 255`
3. `c++` **otra vez** — Pasa del máximo y **da la vuelta**. → `c = 0`

SIN SIGNO
Da la vuelta a 0. Está definido.

CON SIGNO
Desbordar es comportamiento **indefinido**.

DOS ERRORES TÍPICOS

1. `int t = millis();` — `millis()` devuelve `uint32_t`: el `int` se desborda a los **~32 s** en el Uno y a los **~24,8 días** en el ESP32. → ✔ `uint32_t t`
2. `for (uint8_t i = 0; i < 256; i++)` — `uint8_t` nunca llega a 256 (pasa de 255 a 0): bucle infinito. → ✔ `uint16_t i`

4 Pulsador: 3 ideas

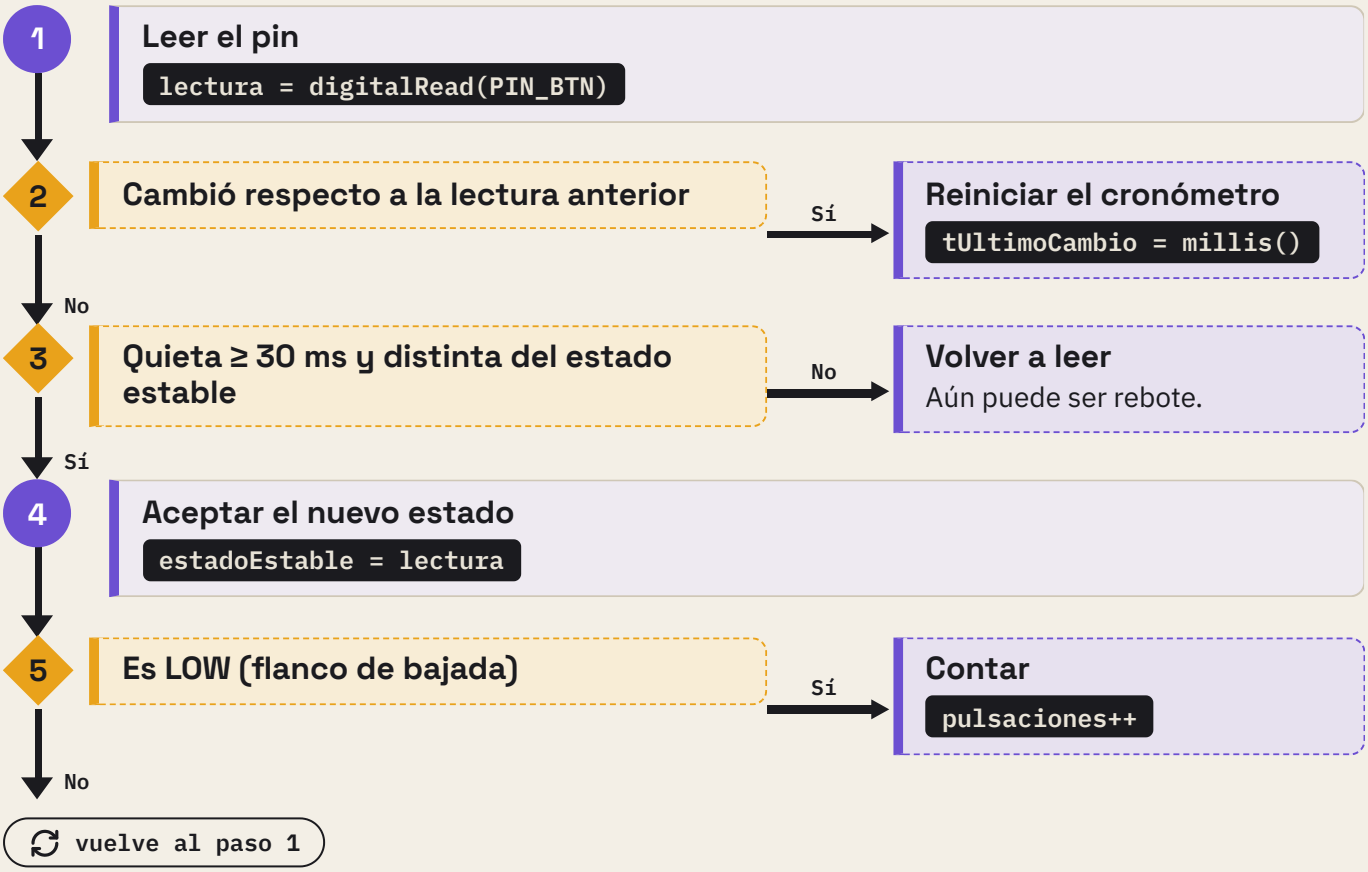
Pull-up
Suelto = HIGH. Presionado = LOW. Con `INPUT_PULLUP`.
`GPIO 34-39: sin pull-up interna`

Rebote
Un botón mecánico da varios cambios falsos en pocos ms.
`esperar 20-50 ms`

Flanco
Actuar solo en el **cambio** HIGH → LOW, no mientras está en LOW.
`1 toque = 1 conteo`

5 Antirrebote + flanco

un toque = un conteo



CÓDIGO: ANTIRREBOTE + FLANCO

```
const uint8_t PIN_BTN = 21;
const uint32_t T_REBOTE = 30;   // ms
bool lecturaAnterior = HIGH, estadoEstable = HIGH;
uint32_t tUltimoCambio = 0;
uint16_t pulsaciones = 0;

void setup() { pinMode(PIN_BTN, INPUT_PULLUP); }

void loop() {
  bool lectura = digitalRead(PIN_BTN);
  if (lectura != lecturaAnterior) {      // cambió: reiniciar
    tUltimoCambio = millis();
    lecturaAnterior = lectura;
  }
  if (millis() - tUltimoCambio >= T_REBOTE && lectura != estadoEstable) {
    estadoEstable = lectura;             // cambio real
    if (estadoEstable == LOW) pulsaciones++;  // flanco de bajada
  }
}
```